iOS SDK Introduction

WeDoBooks SDK Integrator Guide

Setup and Authentication

Goal

Initialize SDK safely, choose backend mode explicitly, and authenticate users before book operations.

Integrator Steps

1. Add SPM dependency from the SDK umbrella repo.
2. Provide a `FirebaseAdapterFactoryType` implementation from FirebaseProvider.
3. Call `WeDoBooksFacade.shared.setup(...)` once per app launch on main actor.
4. Fetch a custom token from your backend and sign in using `userOperations.signIn(with:)`.

Setup Example (Default Firebase App)

```
import WeDoBooksSDK
import FirebaseProvider

@MainActor
func configureSDK() throws {
    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: nil
    )
}
```

Setup Example (Named Firebase App)

Use a custom plist when the SDK should use a non-default Firebase app.

```
import WeDoBooksSDK
import FirebaseProvider

enum SDKSetupExampleError: Error {
    case missingFirebaseConfigFile(String)
}

@MainActor
func configureSDKWithNamedFirebasePlist() throws {
    let plistName = "GoogleService-Info-WDBSDK"
     guard let configPath = Bundle.main.path(forResource: plistName, ofType: "plist")
else {
        throw SDKSetupExampleError.missingFirebaseConfigFile(plistName)
    }

    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
```

```
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: configPath
    )
}
```

Token Acquisition Guidance

- Production: Host app requests token from your backend. Your backend performs backend-to-backend token exchange with the WeDoBooks token endpoint and returns a short-lived custom token to the host app.
- Testing: You can expose a demo token endpoint pattern (for example `https://example-integrator.test/wedobooks/demo-token`) that returns a token for sandbox testing flows.

Security note: Demo token endpoints are for testing and non-production flows only.

Auth Example (Sign In With Custom Token)

```
func signIn(token: String) async {
    let result = await WeDoBooksFacade.shared.userOperations.signIn(with: token)
    if case .failure(let error) = result {
        print("Sign in failed: \(error)")
    }
}
```

Support ID Example (Library Mode)

Users can read their support ID out to your support staff instead of their user id.

```
func loadSupportId() async {
    switch await WeDoBooksFacade.shared.userOperations.getSupportId() {
    case .success(let supportId):
        print("Support ID: \(supportId)")
    case .failure(.userNotReady):
         // The user's profile is still being created, for example just after sign-up.
Retry shortly.
        print("Support ID not ready yet")
    case .failure(.noUserSignedIn):
        print("Sign the user in first")
    case .failure(.unsupportedInStreamingMode):
        print("No support ID for this user")
    case .failure(.underlyingError(let error)):
        print("Could not load support ID: \(error)")
    }
}
```

- Support IDs are only available in `library` mode. In `streaming` mode the call fails with `SupportIdError.unsupportedInStreamingMode`.

Flow Diagram

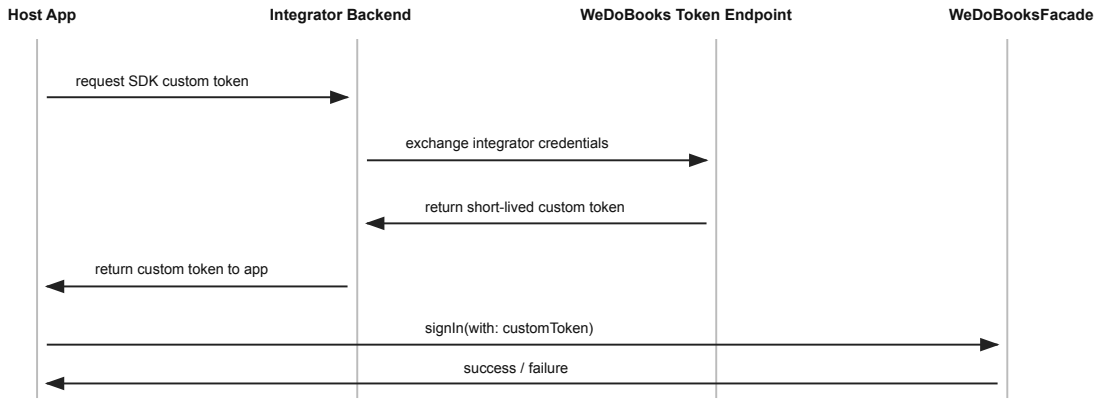


Figure 1: Setup and auth sequence

Optional Demo Token Fetch Example (Testing)

```swift
import Foundation

struct DemoTokenResponse: Decodable {
    let token: String
}

func fetchDemoTokenForTesting(userId: String) async throws -> String {
    let encodedUser = userId.addingPercentEncoding(
        withAllowedCharacters: .urlQueryAllowed
    ) ?? userId

    let baseURL = "https://example-integrator.test/wedobooks/demo-token"
    let url = URL(string: "\(baseURL)?user_id=\(encodedUser)")!
    let (data, _) = try await URLSession.shared.data(from: url)

    return try JSONDecoder()
        .decode(DemoTokenResponse.self, from: data)
        .token
}
```

Notes

- `setup(...)` is intentionally explicit about backend mode.
- `customProgressConfig` is optional in `setup(...)`; omitting it uses `CustomProgressConfig()` (custom progress disabled for both reader and player, so the SDK restores its own saved progress on open). Progress is always tracked regardless of this config.
- Repeated setup attempts throw `SetupError.setupAlreadyCalled`.
- Most namespaces that interact with user data require a signed-in user.

Device Management Example

Use `devicesManager.observeDevices` to monitor devices and `devicesManager.removeDevices(with:)` to remove one or more device ids from the current user's device list.

```
func removeExistingDevice(deviceId: String) async {
        let result = await WeDoBooksFacade.shared.devicesManager.removeDevices(with:
[deviceId])
    if case .failure(let error) = result {
        print("Could not remove device: \(error)")
    }
}
```

Limits and Active Device

The backend currently enforces a per-user device limit of **6 registered devices**. When the limit is reached, the next call to `bookOperations.openCheckout(...)` or `headlessAudioPlayer.loadBook(...)` throws `DeviceManagementError.deviceLimitReached`; call `devicesManager.removeDevices(with:)` to free a slot.

Only one device can be active per user at any time. When another device becomes active, the SDK terminates the local reader/player/headless session and emits `.sessionTakenOver` on `events.sessionInterrupted`.

Cross References

- DocC: GettingStarted.md
- DocC symbol root: `WeDoBooksFacade`

Checkout and Open

Goal

Request/observe user checkouts and present reader/player UI safely.

Recommended Flow

1. Ensure user is signed in.
2. Observe checkouts (`observeCheckouts`) for UI list updates. The publisher never fails; if the checkouts listener fails it emits an empty array, and calling `observeCheckouts` again restarts it.
3. Request missing checkout (`checkoutBook(with:)`) when needed.
4. Present using `openCheckout(_:presentedBy:customCover:)` on main actor — or `openCheckout(id:presentedBy:customCover:)` if you only kept the `Checkout.id`.

Checkout + Open Example

```
func open(isbn: String, presenter: UIViewController) async {
    let result = await WeDoBooksFacade.shared.bookOperations.checkoutBook(with: isbn)
    switch result {
    case .success(let checkout):
        do {
                try await WeDoBooksFacade.shared.bookOperations.openCheckout(checkout,
presentedBy: presenter)
        } catch {
            print("Open failed: \(error)")
        }
    case .failure(let error):
        print("Checkout failed: \(error)")
    }
}
```

Open By Checkout Id

If your app persists `Checkout.id` (for example in a deep link or a local reading list) rather than the `Checkout` value, open by id instead. The SDK matches the id against the user's current checkouts (the same set `observeCheckouts` emits) and then applies the same guards as `openCheckout(_:presentedBy:customCover:)`. An id is only resolvable once the checkouts have been observed, so wait for a first `observeCheckouts` snapshot after sign-in.

```
do {
        try  await  WeDoBooksFacade.shared.bookOperations.openCheckout(id:  checkoutId,
presentedBy: presenter)
} catch OpenCheckoutError.checkoutNotFound {
    // No current checkout has this id — request one via checkoutBook(with:)
} catch {
    print("Open failed: \(error)")
}
```

Notes

- `Checkout.type` decides which UI opens: `.ebook` opens the reader, `.audiobook` and `.podcast` open the audio player. A podcast is handled exactly like an audiobook throughout the SDK (player, download, progress, stats, samples), so grouping the two cases is usually all a host app needs.
- Only one SDK book UI can be open at a time.
- If headless audiobook playback is active, `openCheckout` throws `OpenCheckoutError.aBookIsAlreadyOpen`.
- The audiobook player's more menu includes a Chapters entry that lists the material's chapters (sorted by start position) when the backend provides chapter metadata; selecting a chapter jumps playback to its start. Materials without chapter metadata show a localized "No chapters" empty state. The same chapter metadata is available via `bookOperations.getAudiobookChapters(for:)` (see the headless chapter for pairing it with headless playback).
- The reader's read-aloud uses the device's speech synthesizer. When an EPUB includes the publisher's recorded narration (EPUB 3 media overlays), tapping read aloud first asks which to use — the publisher's narration or a synthetic voice — and the choice lasts until the reader closes. No configuration is needed.
- `openCheckout` can throw `DeviceManagementError.deviceLimitReached`; free a slot via `devicesManager.removeDevices(with:)` and retry.
- Ebook open path may show download/progress UI before reader when local file is missing.
- Opening a book while the user has a *different* audiobook minimized stops that minimized playback first (its position is saved). Reopening the same audiobook resumes the minimized session instead of restarting it. The SDK's audio runtime hosts one book at a time, so a minimized player is not a reason for `openCheckout` to fail — it is simply ended.
- `openCheckout` refuses to open a checkout/loan that is already no longer active or past its `end` date, throwing `OpenCheckoutError.accessExpired`. The check runs before device activation, so a dead checkout never marks this device active or takes over another device's session. Inform the user their access has expired (re-check out the title if the backend allows it). This is the pre-flight counterpart to the mid-session interruption below.
- The SDK also enforces the access deadline locally while a book is open: when the session opens, the SDK schedules a one-shot timer for the checkout's `end`, so the reader/player closes automatically the moment access runs out and `events.sessionInterrupted` emits `SessionInterruptionEvent.accessExpired` (no associated value) — the same case in both streaming and library modes. The deadline also covers a minimized audiobook player, and the reader, player, and headless runtime all re-check when the app returns to the foreground, so a deadline that elapsed while the device was asleep or the app suspended is caught on resume. Sample sessions are exempt. Note that `bookWillClose` accompanies an expiry only when a reader/player was actually presented — an expiring headless or minimized session emits `.accessExpired` alone. Handle it alongside `.sessionTakenOver(activeDevice:)` — dismiss any host UI and inform the user their access has expired.
- The reader supports both portrait and landscape on iPhone and iPad. For rotation to take effect, your host app must permit landscape in its `UISupportedInterfaceOrientations` (iPhone) / `UISupportedInterfaceOrientations~ipad` (iPad); if the app is portrait-locked at the Info.plist level, the reader stays portrait.
- Startup position overrides are available via `bookOperations.setInitialPlayerTimestampSeconds(_:)` and `bookOperations.setInitialReaderCfi(_:)`.
- Override values are sticky across opens until cleared by setting `nil`.
- Reading/listening progress is **always tracked**. `customProgressConfig` only controls where a book *starts*, per format:

- when custom progress is disabled for the format (the default), the SDK restores its own saved bookmark on open. Supplying an initial position is rejected: `openCheckout(...)` throws `OpenCheckoutError.initialPlayerProgressNotAllowed` (audiobook) or `OpenCheckoutError.initialReaderCfiNotAllowed` (ebook).
- when custom progress is enabled for the format (`customProgressConfig.player` / `customProgressConfig.reader`), the SDK starts at the value set via `setInitialPlayerTimestampSeconds(_:)` / `setInitialReaderCfi(_:)`. If none was set, the book starts from the beginning.

Reader Progress And Navigation

The `reader` namespace provides runtime progress observation and navigation helpers while reader UI is open.

```
import Combine
import WeDoBooksSDK

private var cancellables = Set<AnyCancellable>()

func observeReaderState() {
    WeDoBooksFacade.shared.reader.observeBookLoaded()
        .sink { isLoaded in
            print("Reader loaded: \(isLoaded)")
        }
        .store(in: &cancellables)

    WeDoBooksFacade.shared.reader.observeSessionProgress()
        .sink { progress in
            print("Reader progress: \(String(describing: progress))")
        }
        .store(in: &cancellables)
}
```

```
@MainActor
func jumpToMiddleOfBook() async {
    do {
       let cfi = try await WeDoBooksFacade.shared.reader.percentageToCfi(percentage: 0.5)
        try WeDoBooksFacade.shared.reader.goTo(cfi: cfi)
    } catch {
        print("Reader navigation failed: \(error)")
    }
}
```

Important runtime constraints: - `percentageToCfi(...)` and `goTo(cfi:)` require an active reader session. - `percentageToCfi(percentage:sectionIndex:)` throws `chapterNotFound` when section index is invalid for the open book. - `percentageToCfi(...)` and `goTo(cfi:)` can throw `bookNotLoaded` if called before the active reader runtime has fully loaded.

Stored Book Progress

The `bookOperations` namespace returns the user's persisted position for a material, either as a stream or as a one-shot read. `BookProgress` carries the material-specific position, so you can show where the reader actually is instead of a percentage alone.

```
switch await WeDoBooksFacade.shared.bookOperations.bookProgress(for: materialId) {
case .success(let progress):
    switch progress {
    case .audiobook(let audiobook):
        print("At \(audiobook.timestampSeconds)s, \(audiobook.progress) done")
    case .ebook(let ebook):
        print("At cfi \(ebook.cfi ?? "-"), \(ebook.progress) done")
    case .none:
        print("No stored progress yet")
    @unknown default:
        break
    }
case .failure(let error):
    print("Progress read failed: \(error)")
}
```

Use `observeBookProgress(for:)` for the publisher variant; it re-emits as progress sync changes and throws `CheckoutError.noUserSignedIn` when called without a signed-in user.

Notes: - `materialId` and `progress` are available on every case, so reading only the fraction needs no `switch`. - `BookProgress` is a non-frozen enum: switches over it need an `@unknown default` branch. - An audiobook's `timestampSeconds` pairs directly with `HeadlessAudioPlayer.seek(to:)`. - An ebook's `cfi` is an opaque reader locator. There is no stored page number — page counts come from live reader state via `reader.observeSessionProgress()` while reader UI is open.

Samples

`openSample(for:type:presentedBy:)` is the entry point for presenting either the SDK reader or audio player on a sample without an active checkout. Pass `MaterialType.ebook`, `MaterialType.audiobook` or `MaterialType.podcast` as the type (`.podcast` behaves like `.audiobook`). The call is `async throws`: synchronous pre-flight guards (signed-in user, no other SDK book UI open) throw before the first `await`; the URL fetch (and EPUB download for `.ebook`) is awaited inline and any failure is re-thrown to the caller.

```
Task { @MainActor in
    do {
        try await WeDoBooksFacade.shared.bookOperations.openSample(
            for: materialId,
            type: .audiobook,
            presentedBy: presenter
        )
    } catch let error as SampleError {
        // .unauthenticated, .aBookIsAlreadyOpen, .notFound,
        // .urlResolutionFailed, or .underlyingError(...)
        print("Sample open failed: \(error)")
```

```
    }
}
```

`type: .ebook` skips progress persistence, `lastOpened` updates, reading-time statistics, the finish-book button, and bookmarks in full (text-selection menu item, existing bookmark annotations, chapter-overview Bookmarks segment, and bookmark writes are all suppressed). Samples always open at the beginning even when the user has saved progress on the same material's full book.

`type: .audiobook` (and `.podcast`) skips progress persistence, `lastOpened` updates, audible-time statistics, Now Playing info, and remote-control event reception. In the player UI the sleep-timer button, the finish-book button, and the more menu (including its chapters list) are hidden. The audio player UI has no bookmark surface, so no additional bookmark suppression is needed at the UI layer. Samples always start at position 0.

Unlike `openCheckout`, `openSample` does not interact with device management: no backend device activation is performed, this device is not marked as the user's active device, no other device's session is taken over, and `DeviceManagementError` is never thrown from a sample entry point. A running sample session is also not terminated when another device becomes the user's active device.

The `materialId` is not validated locally — it is passed to the backend lookup as-is; identifier hygiene is the caller's responsibility.

For `.ebook`, a transient backend lookup failure is classified as the retryable `SampleError.notFound`; a malformed or empty URL response and every other failure become `SampleError.underlyingError(_:)`. For `.audiobook` and `.podcast`, only a lookup that completes without a URL is `SampleError.notFound`; any lookup failure, including an empty or malformed URL response, is `SampleError.urlResolutionFailed`. A cancelled task propagates `CancellationError` unwrapped. The arbitration check is repeated on the main actor after the URL fetch resolves, so a concurrent SDK open can still throw `SampleError.aBookIsAlreadyOpen` after the await.

The headless `headlessAudioPlayer.loadSample(for:)` and `loadSample(url:materialId:)` APIs remain available for integrators that want to drive audiobook sample playback without presenting SDK UI (see the headless chapter).

Samples from a signed URL

If your app has already obtained a short-lived signed sample URL from the WeDoBooks backend API, use `openSample(url:type:presentedBy:materialId:)` instead of the `materialId` variant. The SDK performs no backend URL lookup of its own: for `.audiobook` the signed URL is streamed directly, and for `.ebook` the EPUB is downloaded from the signed URL into temporary storage (deleted when the reader closes).

No signed-in user is required, for both `.ebook` and `.audiobook`. The SDK performs no backend URL lookup on this path (the audiobook URL is handed straight to the streaming runtime; the ebook EPUB is downloaded directly from the signed URL), so `SampleError.unauthenticated` is never thrown — you can offer samples before the user signs in. The `materialId` variant, by contrast, requires a signed-in user for its backend URL lookup. The only backend read on this path is the material's metadata, when you pass `materialId` and a user is signed in.

```
Task { @MainActor in
    do {
        try await WeDoBooksFacade.shared.bookOperations.openSample(
```

```
            url: signedURLString,
            type: .ebook,
            presentedBy: presenter,
            materialId: materialId // optional
        )
    } catch let error as SampleError {
        // .invalidUrl, .aBookIsAlreadyOpen, .notFound, or .urlResolutionFailed
        print("Sample open failed: \(error)")
    }
}
```

Session behaviour (suppression rules, arbitration re-checks, absence of device activation) is identical to the `materialId` variant. Differences:

- No signed-in user is required — the `materialId` variant throws `SampleError.unauthenticated` without one.
- The only input validation is URL parsing (`SampleError.invalidUrl` for an unparseable string). The optional `materialId` is not validated and is never used to fetch the sample — the signed URL is always the source.
- Treat the signed URL as a short-lived transport URL: the SDK never persists or reuses it across opens, so obtain a fresh one from the backend API per call.
- An expired or rejected signed URL throws `SampleError.urlResolutionFailed` for `.ebook` (the download fails before any UI is presented). For `.audiobook` the URL is handed directly to the streaming runtime without validation, so nothing is thrown — the player presents and the failure surfaces only as a runtime playback error. Validate freshness before opening if you want to avoid presenting a player that cannot stream.
- Pass `materialId` to identify the sampled material: the audio player then shows its cover and, when a user is signed in, the reader and player show its title and author. Without it the player shows the placeholder cover.

Cross References

- DocC: BookOperations.md

Offline and Storage

Goal

Help host apps let users download ebooks and audiobooks, show useful offline state, recover from storage failures, and understand what happens when downloaded material is removed or a user signs out.

Offline support is centered around a `Checkout`. A checkout tells the SDK which material belongs to the signed-in user, and the storage APIs use that checkout to download, inspect, and remove local material.

Offline Model

Downloaded material is scoped to the current signed-in user. The SDK stores downloaded ebook/audio artifacts in app storage, keeps offline bookmarks and related metadata in local preferences, and stores ebook offline keys in the keychain.

Ebooks are always downloaded before they are opened so they are available offline immediately after that and until the material is removed explicitly through the API. Audiobooks and podcasts can be downloaded through a call to the API — both use the same HLS download path. After download has completed successfully then the material can also be opened when offline.

API Quick Reference

Use these APIs to build host UI around the offline model:

- `storageOperations.download(book:)` / `download(checkoutId:)`: Start downloading a checkout for offline use.
- `storageOperations.downloadStatus(book:)` / `downloadStatus(checkoutId:)`: Render download state and progress in host UI.
- `storageOperations.isBookDownloaded(isbn:)`: Check whether the current user has local material for an ISBN.
- `storageOperations.getAllDownloadedBooks()`: List ISBNs with local offline records for the current user.
- `storageOperations.downloadedBytes(for:)`: Bytes one download occupies on disk (`nil` when not downloaded).
- `storageOperations.storageUsage()`: Snapshot of SDK storage on the device: `downloadedBytes` across all users plus the signed-in user's per-book `downloads`.
- `storageOperations.removeDownload(isbn:)`: Remove one downloaded material item for the current user, cancelling its download first if one is running.
- `storageOperations.cancelDownload(isbn:)`: Cancel a download that is in progress; returns whether one was running.
- `storageOperations.removeAll()`: Cancel running downloads and clear SDK local offline/download storage.
- `storageOperations.onError`: Observe asynchronous download failures, each tagged with the failed book's ISBN.
- `storageOperations.downloadStatuses`: Observe asynchronous download status updates.
- `userOperations.signOut()`: Sign out. Downloaded material stays on disk for its owner.

`downloadStatus(book:)` is usually the best API for UI because it can distinguish between in-progress downloads, completed downloads, and material that is not downloaded. The `checkoutId:` variants take a `Checkout.id` instead and throw `StorageOperationError.checkoutNotFound` when no current checkout has that id. `downloadStatuses` publishes snapshots keyed by material identifier (represented as ISBN in current integrator flows).

Most storage APIs require setup to have completed and a user to be signed in. Calls that cannot even start because setup or sign-in is missing throw synchronously. Download failures that happen after a request has started are reported asynchronously through `storageOperations.onError` as a `StorageDownloadError`, which pairs the `StorageOperationError` with the `isbn` of the book that failed. The same error is also attached to that book's `downloadStatuses` entry as its `error`, with `state == .failure`.

Download Lifecycle

Start by acquiring a `Checkout`, for example through `bookOperations.checkoutBook(with:)` or `bookOperations.observeCheckouts()`. Pass that checkout to `download(book:)`.

```
import Combine
import WeDoBooksSDK

private var cancellables = Set<AnyCancellable>()

func prepareOfflineAccess(for checkout: Checkout) {
    WeDoBooksFacade.shared
        .storageOperations
        .onError
        .sink { failure in
            print("Offline download of \(failure.isbn) failed: \(failure.error)")
        }
        .store(in: &cancellables)

    do {
        try WeDoBooksFacade.shared
            .storageOperations
            .download(book: checkout)
    } catch {
        print("Could not start download: \(error)")
    }
}
```

If the material already exists on device for the current user, `download(book:)` throws `StorageOperationError.alreadyDownloaded` instead of starting a new download. Treat this as a benign, already-satisfied state rather than a failure (for example, surface "already available offline" rather than an error).

Use `downloadStatus(book:)` to keep host UI in sync:

```
func refreshOfflineState(for checkout: Checkout) {
    do {
        let status = try WeDoBooksFacade.shared
            .storageOperations
            .downloadStatus(book: checkout)

        switch status.state {
        case .initializing:
```

```
            print("Preparing download")
        case .downloading:
            if let totalBytes = status.totalBytes {
             print("Downloading \(Int(status.progress * 100))% (\(status.downloadedBytes)
of \(totalBytes) bytes)")
            } else {
             print("Downloading \(Int(status.progress * 100))% (\(status.downloadedBytes)
bytes)")
            }
        case .downloaded:
            print("Available offline")
        case .notDownloaded:
            print("Not downloaded")
        case .failure:
            if status.error == .missingDiskSpace {
                print("Download failed: not enough storage")
            } else {
                print("Download failed: \(String(describing: status.error))")
            }
        case .cancel:
            print("Download canceled")
        }
    } catch {
        print("Could not read download status: \(error)")
    }
}
```

Or observe download statuses:

```
func observeDownloadStatuses() {
    WeDoBooksFacade.shared.storageOperations.downloadStatuses
        .sink { statusesByISBN in
            print("Known download statuses: \(statusesByISBN)")
        }
        .store(in: &cancellables)
}
```

`StorageDownloadStatus` is a struct:

- `materialId`: the book's material ID, often the ISBN.
- `materialType`: `.ebook`, `.audiobook` or `.podcast`. The type is remembered with the downloaded material, so it survives an app restart (the synchronous `downloadStatus(book:)` takes it from the checkout). Only a download made before the type was remembered has it inferred from what is on disk in `downloadStatuses`; that cannot tell audiobook and podcast packages apart, so such legacy entries report `.audiobook` until a download notification for the book says otherwise.
- `state`: one of `StorageDownloadState`:
  - `.initializing`: a download has been requested and is being prepared.
  - `.downloading`: a download is active.

  - `.downloaded`: local material exists for the current user and checkout.
  - `.notDownloaded`: local material is absent, not started, or removed.
  - `.failure`: the latest download attempt failed, either before it started (for example blocked on cellular) or while downloading.
  - `.cancel`: the latest download attempt was canceled.
- `progress`: a `0.0...1.0` fraction; `1.0` once downloaded, and the last fraction reached after a failure or cancellation.
- `error`: the `StorageOperationError` behind a `.failure` state, otherwise `nil`. For downloads started with `download(book:)` it is the same error emitted on `onError`. `.missingDiskSpace` means the device ran out of disk space while writing the material.
- `downloadedBytes`: bytes received by the latest download.
- `totalBytes`: the expected size, or `nil` while it is unknown. Audiobook and podcast downloads keep it `nil` while downloading, because HLS reports progress by media time rather than bytes.

Once a download is complete, both counts are the size of the material on disk, so `downloadedBytes == totalBytes` for every `.downloaded` entry, audiobooks included and also after an app restart. Measuring happens off the main thread, so the first snapshot after completion may still carry the transfer counts, the first snapshot after sign-in reports `0` and `nil` for books downloaded in an earlier run, and a follow-up snapshot brings the sizes. After a failure or cancellation the status keeps the progress and counts the download had reached, so you can show where it stopped. Everything resets when a new download of the same book starts.

Cancelling A Download

```
let didCancel = try WeDoBooksFacade.shared.storageOperations.cancelDownload(isbn: checkout.materialId)
```

Cancelling discards the partial file. The book's status becomes `.cancel` and keeps the progress and byte counts it had reached, so you can show where it stopped; `error` is `nil` and nothing is emitted on `onError`. The call returns `false` when nothing was downloading. A download that is still initializing has no transfer to cancel yet: the call returns `false` and the download goes on, so retry once the status is `.downloading`. `removeDownload(isbn:)` cancels a running download before deleting, so you do not need to call both.

To check availability without rendering progress, use:

```
func printDownloadedISBNs() {
    do {
        let isbns = try WeDoBooksFacade.shared
            .storageOperations
            .getAllDownloadedBooks()
        print("Downloaded ISBNs: \(isbns)")
    } catch {
        print("Could not list downloads: \(error)")
    }
}
```

To show how much space downloads use, for example in a storage settings screen:

```swift
func printDownloadSizes() {
    do {
        let usage = try WeDoBooksFacade.shared.storageOperations.storageUsage()
     print("On device: \(ByteCountFormatter.string(fromByteCount: usage.downloadedBytes,
countStyle: .file))")
        for (materialId, bytes) in usage.downloads {
                print("\(materialId): \(ByteCountFormatter.string(fromByteCount: bytes,
countStyle: .file))")
        }
    } catch {
        print("Could not read download sizes: \(error)")
    }
}
```

Sizes cover completed downloads only; a download still in progress is not counted until it finishes.

`downloadedBytes` on the snapshot covers every user who has downloaded on the device, not only the signed-in one, so it is the number to show next to a "Remove all downloads" action backed by `removeAll()`. It can therefore be larger than the sum of `downloads`, which lists only the signed-in user's books. Use `downloadedBytes(for:)` to look up a single book.

Opening Downloaded Material

Open a checkout the same way whether it has been downloaded or not:

```swift
@MainActor
func open(_ checkout: Checkout, from presenter: UIViewController) async {
    do {
        try await WeDoBooksFacade.shared
            .bookOperations
            .openCheckout(checkout, presentedBy: presenter)
    } catch {
        print("Could not open checkout: \(error)")
    }
}
```

For ebooks, offline opening requires both the downloaded file and the SDK's stored offline decryption metadata. If that local information has been removed or is invalid, opening can fail with `OpenCheckoutError.decryptionInformationInvalidOrMissing`. In that case, guide the user to download or open the material again while online.

Reading position is stored in the backend, not on the device. A downloaded ebook always opens offline, but when the saved position cannot be read it opens at the beginning of the book instead of the last read page. Position tracking resumes as soon as the device is back online.

For audiobooks, a streamed (non-downloaded) title cannot be opened while offline: `openCheckout` throws `OpenCheckoutError.notAvailableOffline` and does not present the player. Download the title first (`storageOperations.download(book:)`) to open and listen offline; downloaded audiobooks open from local storage regardless of connectivity.

Disk Space and Download Failures

When a download fails because the device ran out of disk space while writing the material, the SDK detects the out-of-space condition from the underlying OS write error and reports `StorageOperationError.missingDiskSpace` through `storageOperations.onError`. A background ebook transfer that the system drops mid-write arrives without a cause; the SDK then reports `missingDiskSpace` when the device has less free space than the file needs, and `underlyingError` otherwise. Audio downloads carry no expected size, so a dropped audio transfer is always `underlyingError`; its partial package is deleted, although the status keeps the byte count it had reached. Such a drop is never reported as `cancel`; that state only follows `cancelDownload(isbn:)` or `removeDownload(isbn:)`. Any other post-start download failure is reported as `StorageOperationError.underlyingError` instead. (Disk-space classification applies to the asynchronous write path; a rare failure to even create the storage location before the download starts still surfaces as `underlyingError`.)

A practical recovery path is:

1. Tell the user the download could not be completed.
2. Suggest freeing device storage.
3. Let the user retry the download.
4. Refresh `downloadStatus(book:)` after the failure or retry.

```
func handleStorageError(_ error: StorageOperationError) {
    switch error {
    case .setupNotCompleted:
        print("SDK setup has not completed.")
    case .noUserSignedIn:
        print("Sign in before using offline storage.")
    case .downloadNotAllowedOnCellular:
        print("Connect to Wi-Fi or allow cellular downloads.")
    case .alreadyDownloaded:
        print("This book is already available offline.")
    case .missingDiskSpace:
        print("Not enough storage. Free up space and try again.")
    case .checkoutNotFound:
        print("That checkout is not in the current list.")
    case .underlyingError:
        print("The download failed. Please try again.")
    }
}
```

Removing Downloaded Material

Use `removeDownload(isbn:)` to remove a single downloaded item for the current user:

```
func removeOfflineMaterial(isbn: String) {
    do {
        try WeDoBooksFacade.shared
            .storageOperations
            .removeDownload(isbn: isbn)
```

```
    } catch {
        print("Could not remove downloaded material: \(error)")
    }
}
```

After a successful removal, the SDK clears the cached download state for that ISBN. A later `downloadStatus(book:)` call should resolve to `.notDownloaded`.

Opening the same material again after removal may require network access so the SDK can stream or download the material again. For ebooks, the removed offline file and decryption metadata cannot be used after removal.

Use `removeAll()` when the host app needs to clear SDK offline storage explicitly:

```
func clearAllSDKOfflineStorage() {
    WeDoBooksFacade.shared
        .storageOperations
        .removeAll()
}
```

`removeAll()` cancels running downloads first, then clears downloaded artifacts, bookmarks/offline metadata, ebook offline data, easy-access entries, and cached download-status state.

Logout Behavior

`userOperations.signOut()` signs out the current user. It deletes nothing: downloaded material, offline metadata and easy-access entries stay on disk, scoped to the user who created them.

```
func signOut() {
    WeDoBooksFacade.shared
        .userOperations
        .signOut()
    // Downloads stay on disk; `downloadStatuses` emits an empty snapshot until a user
signs in.
}
```

After logout, `downloadStatuses` emits an empty snapshot and in-session download tracking is cleared. When the same user signs in again, their downloads report `downloaded` without a new transfer; another user never sees them. Call `removeAll()` before signing out when the device should be cleared.

Implementation Notes

- Download failures can be synchronous (`throw`) or asynchronous (`onError`).
- Status is exposed as the `StorageDownloadStatus` struct; its `state` is a `StorageDownloadState` of `initializing`, `downloading`, `downloaded`, `notDownloaded`, `failure`, or `cancel`. Exhaustive `switch` statements over `status.state` must handle all six.
- `StorageDownloadStatus` used to be an enum with associated values. Code that switched over it, matched a case pattern, compared `== .failure(...)`, or built a case must switch on `status.state` and read `status.progress`, `status.error`, `status.downloadedBytes` and `status.totalBytes` instead.

`StorageDownloadStatus.FailureReason` is gone: an out-of-disk-space failure is `state == .failure` with `error == .missingDiskSpace`.

- `onError` emits `StorageDownloadError` rather than a bare `StorageOperationError`; switch on its `error` property.
- The `.cancel` state only follows a cancellation: `cancelDownload(isbn:)`, or `removeDownload(isbn:)` when the deletion after its cancel fails. Earlier SDK versions declared the case but never produced it.
- `downloadStatuses` emits the current snapshot immediately for the current user's persisted downloads plus active/known download states, then emits whenever a known book's state or progress changes.
- `downloadStatuses` is scoped to the current signed-in user. Switching users resets in-session download tracker state before the new user's snapshot is emitted. A download the previous user started keeps running for that user and never appears in the new user's statuses; it shows as `downloaded` when its owner signs in again. `cancelDownload(isbn:)` only cancels the signed-in user's own download.
- User sign-out keeps downloaded material on disk; only in-session download tracking is cleared.

Cross References

- DocC: StorageOperations.md
- DocC: BookOperations.md
- DocC: UserOperations.md

Customization

Goal

Adjust SDK behavior and visual integration without forking SDK internals.

Available Areas

- `configuration`: player and reader toggles (`configuration.player`, `configuration.reader`), set on every launch, and `isWifiDownloadOnly`, persisted between launches. `configuration.reader.highlightCallback` keeps your own copy of the user's highlights; see Custom Highlight Storage
- `styling`: light/dark theme override
- `localization`: language selection, string overrides and VoiceOver text overrides (`setAccessibilityLocalizations`)
- `images`: icon and reader image overrides
- `events`: host-side reactions to SDK UI actions

Configuration Example

```
let facade = WeDoBooksFacade.shared

// Book UI: read when a book is opened, not persisted between launches.
// Immutable values: assign a new configuration to change a toggle.
facade.configuration.player = PlayerConfiguration(
    showMinimizeButton: true,
    onTappedMinimizeButton: { isbn in
        showMiniPlayer(for: isbn)
    },
    showFinishBookButton: true,
    showAboutBookButton: true
)
facade.configuration.reader = ReaderConfiguration(
    pullDownTop: false,
    showHeaderAnimation: false,
    showFinishBookButton: true
)

// Download policy: persisted between launches.
facade.configuration.isWifiDownloadOnly = false
```

`showFinishEbookButton`, `showFinishAudiobookButton`, `showAudiobookMinimizeButton`, `showAboutAudioBookButton` and `allowEbookDownloadUsingMobileData` directly on `configuration` are deprecated; they forward to `configuration.player` / `configuration.reader` / `isWifiDownloadOnly` (`allowEbookDownloadUsingMobileData` is the inverse of `isWifiDownloadOnly`).

Localization Override Example

```
typealias LocalizationOverrides = [
    WeDoBooksFacade.Localization.LocalizationKeys: [
```

```
        WeDoBooksFacade.Localization.Language: String
    ]
]

let overrides: LocalizationOverrides = [
    .buttonSave: [.english: "Save"],
    .playerPlaybackRateSpeed: [.english: "Speed"]
]

WeDoBooksFacade.shared.localization.setCustomLocalizations(overrides)
```

Accessibility Labels

The reader and the player support VoiceOver. The texts VoiceOver reads have their own overrides, separate from the visible-text overrides above: `localization.setAccessibilityLocalizations` with `AccessibilityKeys`. They never change visible text, and neither call replaces the other's overrides. Keys prefixed `reader` apply to the reader and keys prefixed `player` to the player; they are independent, so overriding `readerPlay` leaves the player's `playerPlay` unchanged.

Without an override for the current language, VoiceOver reads the SDK's own translation where one exists, and English otherwise (for example `readerPreviousPage`, the reader background colour names, `readerPageNumber`, `playerPlay` and `playerMinimizePlayer`). Provide every language your app supports. The full key list and their English defaults are in the DocC Localization article.

The translated defaults reuse visible SDK strings, so they follow your visible-text overrides: for example, overriding `.playerPlaybackRateSpeed` also changes what VoiceOver reads for `playerPlaybackSpeed`, until you override `playerPlaybackSpeed` itself.

Every reader text VoiceOver reads that comes from the SDK has a key, including sheet titles, the overview and search segments, empty states, settings rows and the bookmark editor. Book content (chapter titles, bookmarks, search results, the book's text) is read as it is. Set the overrides before you open the reader; it applies them when it appears.

VoiceOver users show the reader's pull-down stats header with a three-finger swipe down, which moves VoiceOver to it and reads the book title and its stats (minutes read today and the next chapter), and hide it with a three-finger swipe up. The page indicator also reads the stats after the page number. Both read the minutes read today from `readerMinutesReadToday`, which defaults to the header's visible text, so your `readerHeaderReadMinutesTodayPlus` override applies there too until you override that key. The next chapter reads the header's visible texts, so your `readerHeaderNextChapterNameFallback` and `readerHeaderNextChapterMinuteAbbreviation` overrides apply.

```
WeDoBooksFacade.shared.localization.setAccessibilityLocalizations([
    .readerCloseBook: [.danish: "Luk bog", .english: "Close book"],
    .readerPreviousPage: [.danish: "Forrige side", .english: "Previous page"],
    // Format string: keep both placeholders (current page, page count).
    .readerPageNumber: [.danish: "Side %1$@ af %2$@", .english: "Page %1$@ of %2$@"],
    .playerMinimizePlayer: [.danish: "Minimer afspiller", .english: "Minimize player"]
])
```

Events Example

Finish taps are available both as callbacks on `configuration.player` / `configuration.reader` and as the `events.finishBookTapped` publisher. In the player, the SDK closes the player first; the callback and event follow once it has been dismissed, so it is safe to present your own UI from them.

```
private var cancellables: Set<AnyCancellable> = []

func observeEvents() {
    WeDoBooksFacade.shared.configuration.reader = ReaderConfiguration(
        showFinishBookButton: true,
        onTappedFinishBookButton: { isbn in
            print("Finish ebook tapped for \(isbn)")
        }
    )

    WeDoBooksFacade.shared.events.finishBookTapped
        .sink { isbn in
            print("Finish tapped for \(isbn)")
        }
        .store(in: &cancellables)
}
```

Cross References

- DocC: Configuration.md
- DocC: Styling.md
- DocC: Localization.md
- DocC: Images.md
- DocC: Events.md

Headless Audio and CarPlay

Goal

Use SDK audio runtime without presenting SDK player UI, and optionally integrate transport controls in host app surfaces. Audiobooks and podcasts are both accepted — they are the same runtime.

Headless Flow

1. Ensure setup + sign-in is completed.
2. Load an audiobook or podcast checkout with `headlessAudioPlayer.loadBook(...)`. An ebook checkout is rejected with `.checkoutIsNotAudiobook`.
3. Use `play`, `pause`, `seek`, `setRate` controls.
4. Observe `onStateChange` and `onError` on main queue.
5. Observe `events.sessionInterrupted` to react when the SDK ends the session itself — another device takes over (`.sessionTakenOver`), or the loaded checkout/loan reaches its end date (`.accessExpired`).

`loadBook(...)` also updates the device-local easy-access pointer for the loaded checkout (the same pointer SDK player UI and the reader set), so the host's easy-access widget (`easyAccess.lastOpenedBook()`) reflects the headless book. `loadSample(...)` does not.

Start position and custom progress

`loadBook`'s `startPosition` is optional and honors the `player` flag of the `CustomProgressConfig` you pass to `setup(...)`, mirroring `openCheckout(...)`:

- **Custom player progress enabled** (`CustomProgressConfig(player: true)`): you own the position — the supplied `startPosition` is applied (or playback starts from the beginning when omitted), and the SDK does not restore its saved progress.
- **Custom player progress disabled** (the default): the SDK restores its own saved progress. Supplying a non-`nil` `startPosition` throws `HeadlessAudiobookError.startPositionNotAllowed`; omit it (or pass `nil`) to resume from the saved progress.

Progress is always tracked regardless of this flag.

Switching books

Calling `loadBook(...)` (or `loadSample(...)`) while another audiobook is loaded — playing or paused — stops the current session, persists its position, then loads the requested audiobook. Loading the ISBN already loaded throws `HeadlessAudiobookError.sameBookAlreadyLoaded`. `HeadlessAudiobookError.audiobookAlreadyPlaying` is no longer thrown (retained only for source compatibility).

`loadBook(...)` also refuses a checkout/loan that is no longer active or past its `end` date, throwing `HeadlessAudiobookError.accessExpired` (checked before device activation, so no device is marked active). This mirrors `openCheckout`'s pre-flight guard and is the load-time counterpart to the mid-session `.accessExpired` interruption.

If you only kept the `Checkout.id`, use `loadBook(checkoutId:startPosition:customCover:recoversWhenBackOnline:)`. It throws `HeadlessAudiobookError.checkoutNotFound` when no current checkout has that id, and otherwise behaves exactly like `loadBook(book:)`.

Headless Example

```
// Default config (custom player progress disabled): omit `startPosition` so the SDK
// resumes from its saved progress. Pass a `startPosition` only when you enabled
// `CustomProgressConfig(player: true)` in `setup(...)`.
func startHeadless(_ checkout: Checkout) async {
    do {
        try await WeDoBooksFacade.shared
            .headlessAudioPlayer
            .loadBook(book: checkout)

        try WeDoBooksFacade.shared
            .headlessAudioPlayer
            .play()
    } catch let error as HeadlessAudiobookError {
        // e.g. .startPositionNotAllowed (startPosition supplied while custom
                                                //      player      progress      is
disabled), .sameBookAlreadyLoaded, .checkoutIsNotAudiobook, .accessExpired
        print("Headless load rejected: \(error)")
    } catch let error as DeviceManagementError {
        // device activation failure, e.g. .deviceLimitReached
        print("Headless activation failed: \(error)")
    } catch {
        print("Headless failed: \(error)")
    }
}
```

CarPlay Example

```
func wireNowPlaying() {
    _ = WeDoBooksFacade.shared.carPlay.nowPlayingInfo
        .sink { info in
            print("Now playing info changed: \(String(describing: info))")
        }
}
```

Now Playing payload

The SDK keeps `MPNowPlayingInfoCenter` up to date for the loaded audiobook and mirrors the same dictionary on `carPlay.nowPlayingInfo`. It carries the book-level timeline that the lock screen, Control Center, CarPlay and AirPlay receivers render:

Key	Value
`MPNowPlayingInfoPropertyElapsedPlaybackTime`	Current position in the book, in seconds
`MPMediaItemPropertyPlaybackDuration`	Total book duration, in seconds
`MPNowPlayingInfoPropertyPlaybackProgress`	Progress fraction, `0...1`

Key	Value
`MPNowPlayingInfoPropertyPlaybackRate` / `MPNowPlayingInfoPropertyDefaultPlaybackRate`	Current playback speed (rate is `0` while paused)
`MPMediaItemPropertyTitle` / `MPMediaItemPropertyArtist`	Book title and author
`MPMediaItemPropertyArtwork`	Cover artwork (custom cover if you supplied one)

The duration and progress keys are absent until the asset reports a usable duration, so read them as optionals rather than assuming they are always present. Sample sessions publish `nil` instead of a dictionary.

```
func observeSessionInterruption() {
    _ = WeDoBooksFacade.shared.events.sessionInterrupted
        .sink { event in
            switch event {
            case .sessionTakenOver(let activeDevice):
                print("Session moved to device: \(activeDevice)")
            case .accessExpired:
             // The loaded book's checkout/loan reached its end date; the headless session
                    // was torn down. Update host UI and inform the user their access
has expired.
                print("Access expired")
            }
        }
}
```

Sample Playback

`loadSample(for:)` loads an audiobook sample into the headless runtime. It takes a raw `materialId` (not a `Checkout`) and never accepts a caller-supplied start position; samples always begin at 0. The material's metadata is loaded first, so `loadedBook()` carries its title and author; if it can't be loaded, the sample loads without them.

```
func startSample(materialId: String) {
    Task { @MainActor in
        do {
         try await WeDoBooksFacade.shared.headlessAudioPlayer.loadSample(for: materialId)
            try WeDoBooksFacade.shared.headlessAudioPlayer.play()
        } catch let error as HeadlessAudiobookError {
            // .setupNotCompleted, .noUserSignedIn,
            // .playerControlledByPresentedSDKUI, or .sameBookAlreadyLoaded
            // (a different loaded book is stopped and replaced instead of throwing)
            print("Sample load rejected: \(error)")
        } catch {
            print("Sample load failed: \(error)")
        }
```

```
    }
}
```

Sample sessions suppress Now Playing info, remote control event registration, progress writes, last-opened updates, the easy-access pointer write (only `loadBook` sets it), and reading-time stats.

Unlike `loadBook`, `loadSample` does not interact with device management: no backend device activation is performed, this device is not marked as the user's active device, no other device's session is taken over, and `DeviceManagementError` is never thrown from a sample entry point. A loaded sample session is also not terminated when another device becomes the user's active device.

The URL fetch is asynchronous; fetch failures surface on `onError` as `HeadlessAudiobookError.underlyingError(SampleError)`. A transient backend lookup failure is classified as the retryable `SampleError.notFound`; the loaded slot is reset on failure so subsequent `loadSample` / `loadBook` calls aren't blocked.

Sample Playback from a Signed URL

If your app has already obtained a short-lived signed sample URL from the WeDoBooks backend API, use `loadSample(url:materialId:)` instead of the `materialId` variant. The SDK performs no backend URL lookup — the signed URL is handed directly to the audio runtime.

```
Task { @MainActor in
    do {
        try await WeDoBooksFacade.shared.headlessAudioPlayer.loadSample(
            url: signedURLString,
            materialId: materialId // optional
        )
        try WeDoBooksFacade.shared.headlessAudioPlayer.play()
    } catch let error as HeadlessAudiobookError {
        // .invalidUrl, .setupNotCompleted, .playerControlledByPresentedSDKUI,
        // or .sameBookAlreadyLoaded (only when a materialId is passed)
        print("Sample load rejected: \(error)")
    } catch {
        print("Sample load failed: \(error)")
    }
}
```

Session suppression rules and the device-management exemption are identical to `loadSample(for:)`. Differences:

- A `url` string that cannot be parsed into a `URL` throws `HeadlessAudiobookError.invalidUrl`. A parseable URL is then accepted as-is. The optional `materialId` is not validated and is never used to fetch the sample — the signed URL is always the source.
- `loadedBook()` returns the synthetic sample checkout, carrying the `materialId` you passed and, when a user is signed in, the material's title and author. The sample then counts as that material being loaded: `loadBook(...)` and both `loadSample` variants for the same material throw `HeadlessAudiobookError.sameBookAlreadyLoaded` until you load something else or call `bookOperations.stopAudioPlayer()`. Without a `materialId` the checkout carries the signed URL as its

id; the error is never thrown, and every call stops the current session (persisting its position first) and replaces it.
- Treat the signed URL as a short-lived transport URL: the SDK never persists or reuses it across loads, so obtain a fresh one from the backend API per call.
- There is no async URL fetch, so no `SampleError` classification applies. An expired or rejected signed URL surfaces as a runtime failure on `onError` or a session stuck in `.buffering` — obtain a fresh signed URL and reload (after `bookOperations.stopAudioPlayer()` if you passed a `materialId`).

Beyond load-time failures, `onError` also reports a runtime death: if streamed playback runs past the buffered audio while offline (played to the end of the buffer, or seeked/skipped outside it), it emits `HeadlessAudiobookError.streamDiedOffline`. The runtime reverts to a loading state (`onStateChange` → `.buffering`). By default (`loadBook(..., recoversWhenBackOnline: true)`) the SDK rebuilds the stream on the next reconnect — playback does not auto-resume, so prompt the user to `play()`. Pass `recoversWhenBackOnline: false` to `loadBook` to handle recovery yourself instead. Downloaded titles never hit this.

Audiobook Chapters

`bookOperations.getAudiobookChapters(for:)` fetches a material's chapter list — the same metadata backing the Chapters menu in SDK Player UI. It lives on `bookOperations` (not the headless namespace) but pairs naturally with headless playback. It is a read-only fetch. It requires a signed-in user.

```
let chapters = try await WeDoBooksFacade.shared
    .bookOperations
    .getAudiobookChapters(for: materialId)

for chapter in chapters {
    print("\(chapter.title) starts at \(chapter.offsetSeconds)s")
}
```

Chapters are sorted by `offsetSeconds` ascending. A material without chapter metadata returns an empty array. The offsets pair directly with `seek(to:)` to jump playback to a chapter start, and with `currentPosition()` to derive the current chapter. Calling without a signed-in user throws `CheckoutError.noUserSignedIn`; fetch failures throw `CheckoutError.underlyingError(_)`.

Notes

- Headless and SDK Player UI share the same underlying runtime.
- Runtime ownership collisions are surfaced as deterministic errors.
- Session takeover on another device terminates the active SDK reading/playback session (reader UI, player UI, or headless) and emits `events.sessionInterrupted` with `.sessionTakenOver`.
- The headless runtime also enforces the checkout/loan deadline: `loadBook(...)` schedules a one-shot timer for the loaded checkout's `end`, so when that instant arrives the session is torn down and `events.sessionInterrupted` emits `.accessExpired` (no associated value) — cancelled automatically if the book is unloaded or replaced first. The runtime additionally re-checks when the app returns to the foreground, which catches a deadline that came due while the device was asleep. Sample sessions are exempt (`loadSample(...)` never schedules one). A headless expiry emits `.accessExpired` on its own; `bookWillClose` is not sent, because no SDK book UI was presented.
- On session activation, SDK registers this device before marking it active; if the device already exists in backend registry, SDK still marks it active.

- CarPlay controls are non-throwing helpers over the shared runtime.

Cross References

- DocC: HeadlessAudioPlayer.md
- DocC: CarPlay.md

Error Handling and Troubleshooting

Goal

Provide quick diagnosis paths for common setup/auth/open/storage/headless failures.

Error Families

- `SetupError`: setup lifecycle and reader initialization.
- `SignInError`: auth and token/factory integration issues.
- `SupportIdError`: support ID lookup (library mode only). `.noUserSignedIn` (send the user to login), `.userNotReady` (the user's profile does not exist yet, for example just after sign-up — retry shortly), `.unsupportedInStreamingMode`, `.underlyingError(_:)` (let the user try again).
- `CheckoutError` / `OpenCheckoutError` / `DeviceManagementError`: checkout/open/presentation and activation failures.
- `StorageOperationError`: offline and download issues. Asynchronous failures on `storageOperations.onError` arrive wrapped in a `StorageDownloadError` that also carries the failed book's `isbn`.
- `HeadlessAudiobookError`: headless runtime state and arbitration errors.
- `HighlightError` (on `reader.onHighlightError`): `.callbackFailed(_:)` (your `HighlightCallback` threw; the reader stays open), `.cfiTooLong` (a highlight couldn't be saved). See Custom Highlight Storage.
- `SampleError`: sample reader/playback failures. `.invalidUrl` (the signed-URL variant's `url` string could not be parsed into a `URL`), `.unauthenticated` (no signed-in user — material-id entry points only; the signed-URL UI variant requires no signed-in user), `.aBookIsAlreadyOpen` (collision with active SDK UI / headless playback), `.notFound` (backend missing or transient internal failure — retryable), `.urlResolutionFailed` (material-id audio — audiobook or podcast — lookup, or signed-URL ebook download, failed — for example an expired/rejected signed URL), `.underlyingError(_:)` (non-retryable failure path). Additive — host apps with exhaustive `switch` handling without `@unknown default` must include this case.

Troubleshooting Matrix

Use this as a quick lookup when mapping SDK errors to host-app diagnostics:

- `setupAlreadyCalled`: setup executed more than once. First check app lifecycle and singleton setup location.
- `firebaseNotConfigured`: Firebase app config is missing or invalid. First check adapter factory and config plist path.
- `noUserSignedIn`: user-dependent API was called before sign-in. First check the auth token flow and sign-in result handling.
- `aBookIsAlreadyOpen`: runtime ownership collision. First check for existing SDK player/reader UI or headless playback.
- `notAvailableOffline`: `openCheckout` was called for a streamed audiobook that isn't downloaded while the device is offline — the stream can't be fetched, so the player isn't presented. Offer a download (`storageOperations.download(book:)`) or ask the user to reconnect; use `isBookDownloaded(isbn:)` to decide which. Downloaded audiobooks open normally offline.
- `checkoutNotFound`: an id-based API (`openCheckout(id:)`, `download(checkoutId:)`, `downloadStatus(checkoutId:)`, `loadBook(checkoutId:)`) found no current checkout with that id. Either the checkout has ended (request a fresh one via `checkoutBook(with:)`) or the checkouts have not been observed yet — wait for a first `observeCheckouts` snapshot after sign-in.

- `observeCheckouts` suddenly emits an empty array while the user has checkouts: the checkouts listener failed (the failure is on `WeDoBooksFacade.shared.errors`). Call `observeCheckouts` again to restart the listener and resubscribe your UI.
- `accessExpired`: `openCheckout` was called for a checkout/loan that is no longer active or whose `end` date has already passed — the SDK refuses to open it (checked before device activation, so no device is marked active). Inform the user their access has expired and re-check out the title if the backend allows it. This is the open-time counterpart to the mid-session `SessionInterruptionEvent.accessExpired` interruption.
- Headless `streamDiedOffline` (emitted on `headlessAudioPlayer.onError`): streamed playback ran past the buffered audio while offline (played to the end of the buffer, or seeked/skipped outside it). The runtime reverts to a loading state (`onStateChange` → `.buffering`) and rebuilds the stream on the next reconnect; playback does not auto-resume, so prompt the user to `play()`. Downloaded titles never hit this.
- `deviceLimitReached`: device activation failed during open. The backend currently enforces a limit of 6 registered devices per user, and only one device can be active at a time. First inspect the user's device list via `devicesManager.observeDevices` and call `removeDevices(with:)` to free a slot.
- `downloadNotAllowedOnCellular`: cellular download policy blocked the request. First check `configuration.isWifiDownloadOnly`.
- `alreadyDownloaded`: `storageOperations.download(book:)` was called for material already present on device for the current user. This is benign; check `isBookDownloaded(isbn:)` / `downloadStatus(book:)` before offering a download action.
- `missingDiskSpace`: a storage-initiated download failed because the device ran out of disk space while writing the material (emitted on `storageOperations.onError`). Prompt the user to free storage, then let them retry the download and refresh `downloadStatus(book:)`. Additive case — host apps with exhaustive `switch` handling over `StorageOperationError` without `@unknown default` must include it. Every other post-start download failure remains `underlyingError`.
- `bookNotLoaded`: reader navigation/progress conversion was called before active reader runtime finished loading. First check `reader.observeBookLoaded()` before calling `reader.percentageToCfi(...)`/ `reader.goTo(cfi:)`.

Diagnostics Checklist

1. Confirm `setup(...)` succeeded this launch.
2. Confirm expected backend mode (`streaming` or `library`).
3. Confirm current user id is non-nil before user-scoped APIs.
4. Confirm runtime ownership before open/headless transitions.
5. Confirm network/download policy assumptions.

Observing Background Errors (support and diagnostics only)

`WeDoBooksFacade.shared.errors` is a firehose that mirrors Firebase-layer failures (every cloud-function callable failure, Firestore background-listener failures, and failures in the SDK's own highlight storage). It exists **only for support and diagnostics** — subscribe to forward what it emits to your logging, crash-reporting, or support tooling:

```
WeDoBooksFacade.shared.errors
    .receive(on: DispatchQueue.main)
    .sink { error in /* forward to logging / crash reporting */ }
    .store(in: &cancellables)
```

It is **not** a programmatic error channel: do not drive error recovery, retries, control flow, or user-facing UI from it — the typed errors returned by the individual APIs above are the only caller-actionable channel.

The stream is replay-less (subscribe early so you don't miss any), is delivered on the thread the error was reported on (typically a background queue — apply `.receive(on:)` yourself if your logging requires the main thread), and emits `any Error` — treat each error as opaque (log it) rather than switching on its concrete type. It is a superset of, not a replacement for, the typed errors returned by the individual APIs above — a callable failure is mirrored here *and* returned by its typed API, so dedupe if you log from both.

Cross References

- DocC: ErrorHandling.md

History

Goal

Display a user's reading/listening history, check per-material status, and add/complete/remove entries programmatically.

Requirements

- SDK configured with `.streaming` mode. All `historyOperations` methods return `.failure(.unsupportedInLibraryMode)` in `.library` mode.
- A signed-in user. Without one all methods return `.failure(.noUserSignedIn)`.

Listing History

```
let result = await WeDoBooksFacade.shared.historyOperations.list(
    filter: .all,         // .all | .inProgress | .completed
    sort: .newestActivity, // .newestActivity | .oldestActivity
    after: nil,           // pass HistoryCursor from previous page for pagination
    limit: 25
)
switch result {
case .success(let page):
    for item in page.items {
        print("\(item.title) — \(item.status)")
    }
case .failure(let error):
    print("History list failed: \(error)")
}
```

Pagination

When `page.nextCursor` is non-nil, more entries exist. Pass the cursor back as `after:` to load the next page.

```
var cursor: HistoryCursor? = nil
var allItems: [HistoryItem] = []

repeat {
    let result = await WeDoBooksFacade.shared.historyOperations.list(
        after: cursor,
        limit: 25
    )
    guard case .success(let page) = result else { break }
    allItems.append(contentsOf: page.items)
    cursor = page.nextCursor
} while cursor != nil
```

> **Note:** `HistoryCursor` is session-scoped (snapshot-based). Discard it on app restart; it is not serializable across launches.

Checking a Single Material's Status

Use `entry(for:)` to show an "In Progress" or "Completed" badge on a product page. Returns `nil` when the material is not in history.

```
let result = await WeDoBooksFacade.shared.historyOperations.entry(for: materialId)
if case .success(let item) = result, let item {
    print("Status: \(item.status), progress: \(item.progress)")
}
```

Adding an Entry Manually

`add(materialId:)` creates a Completed entry. Fails with `.alreadyInHistory` when the material is already present, or `.materialNotFound` when the material does not exist in the catalogue.

```
let result = await WeDoBooksFacade.shared.historyOperations.add(materialId: isbn)
switch result {
case .success:
    print("Added to history")
case .failure(.alreadyInHistory):
    print("Already in history — no action needed")
case .failure(.materialNotFound):
    print("Material not found")
case .failure(let error):
    print("Add failed: \(error)")
}
```

Marking an Entry Completed

Idempotent. Returns `.notInHistory` if the material has no history entry.

```
let result = await WeDoBooksFacade.shared.historyOperations.markCompleted(materialId:
isbn)
if case .failure(let error) = result {
    print("Mark completed failed: \(error)")
}
```

Removing an Entry

Hard-deletes the history entry. Reading progress (`book_progress`) is preserved — if the user re-opens the book, the backend creates a fresh history entry. Returns `.notInHistory` if no entry exists.

```
let result = await WeDoBooksFacade.shared.historyOperations.remove(materialId: isbn)
if case .failure(let error) = result {
    print("Remove failed: \(error)")
}
```

Notes

- `progress` is a `Double` in the range 0–1. Compute percentage, time remaining, or pages remaining in the host app from `progress` combined with `durationSeconds` (audiobook or podcast) or `wordCount` (ebook).

Error Reference

Error	Trigger
`.noUserSignedIn`	No authenticated user
`.unsupportedInLibraryMode`	SDK in `.library` mode
`.alreadyInHistory`	`add` — material already in history
`.materialNotFound`	`add` — material not in catalogue
`.notInHistory`	`markCompleted` / `remove` — no history entry
`.underlyingError(Error)`	Network or Firestore failure

Cross References

- DocC: History.md

Reservations

Goal

Let users reserve titles that are not immediately loanable, manage their reservation queue, and accept or cancel reservation offers when a held title becomes available — one-shot and via live-updating streams.

Requirements

- SDK configured with `.library` mode. All `reservationOperations` async methods return `.failure(.unsupportedInStreamingMode)` in `.streaming` mode, and the `observe…` accessors throw `ReservationError.unsupportedInStreamingMode`.
- A signed-in user for **all** operations. Without one, the async methods return `.failure(.noUserSignedIn)` and the `observe…` accessors throw `ReservationError.noUserSignedIn` (immediately, at call time).

Concepts

- **Reservation queue** — a `Reservation` is the user's place in line for a title. `reserveBook(isbn:)` may create a loan immediately (`canLoan == .loanable`) or queue the user (`canLoan == .reservable`).
- **Reservation offer** — when a held title becomes available, the backend creates a `ReservationOffer` with an acceptance window (`expiresAt`). The user accepts it (creating a loan), cancels it, or lets it expire.

Reserving and Removing

```
let reserve = await WeDoBooksFacade.shared.reservationOperations.reserveBook(isbn: isbn)
switch reserve {
case .success(let response):
     // response.canLoan: .loanable (loan created now) | .reservable (queued) | quota/
unavailable reasons
    print("Reserve outcome: \(response.canLoan)")
case .failure(let error):
    print("Reserve failed: \(error)")
}

let remove = await WeDoBooksFacade.shared.reservationOperations.removeReservation(isbn:
isbn)
if case .success(let removed) = remove {
    print("Removed: \(removed)")
}
```

Listing Reservations

`getReservations()` returns the user's reservations, wish-list entries included, sorted by loan date. For live UI, subscribe to `observeReservations()`.

```
let result = await WeDoBooksFacade.shared.reservationOperations.getReservations()
if case .success(let reservations) = result {
    for reservation in reservations {
        print("\(reservation.materialId) — expected by \(reservation.loanDate)")
    }
}
```

```
let cancellable = try WeDoBooksFacade.shared.reservationOperations
    .observeReservations()
    .receive(on: DispatchQueue.main)
    .sink { reservations in /* update UI */ }
```

On a product page, `observeReservation(forMaterialId:)` emits the user's reservation for that material, or `nil` when they are not queued for it.

Queue position

`Reservation.queuePosition` is the user's place in line — entries ahead plus one, so the head of the queue is `1`. It is `nil` until the backend has written a position; show no number in that case rather than `0`. The number moves as people ahead borrow, cancel or let an offer lapse, so drive it from `observeReservations()` rather than polling, and use "about" language if the design allows — it can trail a change by a few seconds.

```
if let position = reservation.queuePosition {
    label.text = "No. \(position) in the queue"
} else {
    label.text = nil
}
```

Position `1` is not "ready to borrow". Readiness is signalled by an active `ReservationOffer` (next section); the offer holder still occupies position `1` for everyone behind them.

Reservation Offers

```
let offers = await WeDoBooksFacade.shared.reservationOperations.getReservationOffers()
guard case .success(let offers) = offers, let offer = offers.first else { return }

let accept = await WeDoBooksFacade.shared.reservationOperations.acceptReservationOffer(offerId: offer.id)
switch accept {
case .success(let response) where response.success:
    print("Loan created: \(response.loanId ?? "-")")
case .success(let response):
    print("Not accepted (quota: \(String(describing: response.quotaReject)))")
case .failure(let error):
    print("Accept failed: \(error)")
}
```

To decline an offer, remove its underlying reservation — this cancels the offer and drops the queue entry:

```
_ = await WeDoBooksFacade.shared.reservationOperations.removeReservation(isbn: offer.materialId)
```

Live variants — `observeReservationOffers()`, `observeReservationOffer(offerId:)`, and `observeReservationOffer(forMaterialId:)` — emit updates as offers change, and `nil`/empty while an

offer is not active (expired or consumed). Use `observeReservationOffer(forMaterialId:)` to show an "Available now" banner on a title page.

Notes

- The `observe…` publishers never fail. Underlying listener failures surface on `WeDoBooksFacade.shared.errors` for diagnostics; the stream goes quiet: it emits nothing further until the auth state changes, so call the `observe…` accessor again after handling the error; re-subscribing to the same publisher only replays its last value.
- Records missing a required field are dismissed rather than surfaced with placeholder values; each is reported on `WeDoBooksFacade.shared.errors`.
- Publishers are delivered on a background queue — apply your own `.receive(on:)` before touching UI.
- Each `observe…` call attaches its own listener; the returned publisher is created fresh per call and is not shared between calls. Subscribing to the same stream from several places opens several listeners — multicast a single subscription (Combine's `.share()` / `.multicast(_:)`) if you want to fan out to multiple sinks from one listener.

Error Reference

Error	Trigger
`.unsupportedInStreamingMode`	SDK in `.streaming` mode (reservations are library-only)
`.noUserSignedIn`	No authenticated user
`.operationFailed(Error)`	Callable/network/Firestore failure

Cross References

- DocC: Reservations.md

Custom Highlight Storage

Goal

Keep your own copy of the highlights the user makes in the SDK ebook reader: get told when the user saves or deletes one, and optionally supply the highlights the reader shows. The SDK always keeps the user's highlight changes in its own storage as well.

Requirements

- `setup(...)` has completed and a user is signed in.
- Pass the callback as `highlightCallback` in the `ReaderConfiguration` you assign to `configuration.reader`, before opening an ebook. The reader reads it when the book opens; a book that is already open keeps the callback it opened with. Like the rest of `configuration`, it is not persisted, so set it on every launch.
- Notes are passed in plain text; protecting them is up to your implementation.

How It Works

1. **Implement `HighlightCallback`**: `didSave(_:)` and `didDelete(_:)`, plus `loadHighlights(materialId:)` if the reader should show your own highlights.
2. **Register it** with `configuration.reader = ReaderConfiguration(highlightCallback: ...)`, alongside your other reader values.
3. **Open an ebook** with `openCheckout(...)`. `loadHighlights(materialId:)` is called once. The reader shows the array it returns, even an empty one. If it returns `nil` (the default implementation), the reader shows the SDK's stored highlights, or none if they can't be loaded.
4. **Handle saves and deletes.** The reader updates the page first, then calls `didSave(_:)` after the user creates or edits a highlight, or `didDelete(_:)` after the user deletes one. Calls arrive one at a time, in the order the user made the changes, and may run off the main thread.
5. **Surface failures.** Errors you throw arrive on `reader.onHighlightError` as `HighlightError.callbackFailed(_:)`. Failures in the SDK's own highlight storage are reported on `WeDoBooksFacade.shared.errors` instead.

Example

```
final class MyHighlightStore: HighlightCallback {
    func loadHighlights(materialId: String) async throws -> [Highlight]? {
        try await myStore.highlights(materialId: materialId)
    }

    // `id` stays the same across edits, so upserting by it replaces the earlier version.
    func didSave(_ highlight: Highlight) async throws {
        try await myStore.upsert(highlight)
    }

    func didDelete(_ highlight: Highlight) async throws {
        try await myStore.delete(id: highlight.id)
    }
}

let facade = WeDoBooksFacade.shared
```

```
facade.configuration.reader = ReaderConfiguration(highlightCallback: MyHighlightStore())

facade.reader.onHighlightError
    .sink { error in
        switch error {
        case .callbackFailed(let underlying):
            log("Highlight sync failed: \(underlying)")
        case .cfiTooLong:
            showToast("This highlight is too long to save.")
        }
    }
    .store(in: &cancellables)
```

To keep your own copy while the reader shows the SDK's stored highlights, leave out `loadHighlights(materialId:)`; its default implementation returns `nil`.

Highlight Fields

Field	Meaning
`id`	Stable identifier; the SDK assigns it when the user creates the highlight, and it stays the same across edits
`materialId`	The book the highlight belongs to
`cfi`	Location of the highlighted range in the book
`selectionText`	The highlighted text; the SDK keeps only the start of a very long selection
`color`	`HighlightColor`: `.yellow`, `.green` or `.pink`
`note`	The user's note, in plain text
`sectionIndex`	The reader's table-of-contents entry, as set by the SDK; pass back the value from `didSave(_:)` unchanged

Loading All Notes

By default the reader loads a note only when it is shown (when the user opens the highlight, or expands its chapter in the table of contents), which keeps opening a book fast however many notes it has. Until then, the reader's search can't find the note. Pass `loadAllHighlightNotes: true` to `ReaderConfiguration` to load every note in the background when an ebook opens. Notes from `loadHighlights(materialId:)` always show right away.

Gotchas

Symptom	Cause	What to do
Your callback is never called	It was set after the book opened	Assign the `ReaderConfiguration` with

Symptom	Cause	What to do
		`highlightCallback` at launch, before opening a book
Your callback stops being called	A later `configuration.reader = ReaderConfiguration(...)` left out `highlightCallback`, so it went back to `nil`	Pass `highlightCallback` in every `ReaderConfiguration` you assign
Changes made in the reader don't show when the book is reopened	`loadHighlights(materialId:)` returns your own list, and the reader shows that list instead of the SDK's	Keep that list in sync through `didSave(_:)` and `didDelete(_:)`, or return `nil` to show the SDK's stored highlights
Duplicate highlights after an edit	`didSave(_:)` is called for edits too	Key your storage by `Highlight.id`, so an edit replaces the earlier version
Editing a highlight from your list adds a second copy to the SDK's storage, or deleting it changes nothing	Its `id` didn't come from the SDK, so the SDK writes under that id instead of the original (an empty `id` or one containing `/` is rejected and reported on `WeDoBooksFacade.shared.errors`)	Store the `id` from `didSave(_:)` and pass it back unchanged
Highlights you return are missing from the reader's table of contents	Their `sectionIndex` matches no table-of-contents entry	Store the `sectionIndex` from `didSave(_:)` and pass it back unchanged
A saved highlight stays drawn although your callback threw	The reader updates the page first, then calls your callback	Handle `HighlightError.callbackFailed(_:)` on `reader.onHighlightError`
`HighlightError.cfiTooLong`, and the highlight isn't drawn	The highlighted range's location is too long for the SDK's storage	Tell the user the highlight couldn't be saved; it isn't stored or passed to `didSave(_:)`
No highlights for a whole reading session	`loadHighlights(materialId:)` threw, or it returned `nil` and the SDK's stored highlights couldn't be loaded	Check `reader.onHighlightError` for your error and `WeDoBooksFacade.shared.error` for the SDK's
No highlights in a free preview	Samples keep no highlights and never call your callback	Nothing to do
Several highlights on the same text after switching from your own list back to the SDK's	The SDK keeps writing to its own storage while your list is shown but never merges the two; highlights your store re-keyed or left out come back	Key your storage by `Highlight.id` and keep every highlight `didSave(_:)` hands you

Cross References

- DocC: Configuration.md
- DocC: ErrorHandling.md